

HW7

Problem 1

Part A

```
#libraries
library(nnet)
library(lmtest)
#load data
data <- read.table("policestops.txt",header=TRUE)
#fit the four models
me <- glm(mystops ~ 1, family = poisson(link = "log"), data = data)
met <- glm(mystops ~ factor(myeth), family = poisson(link = "log"), data = data)
mpre <- glm(mystops ~ factor(myprecinct), family = poisson(link = "log"), data = data)
mtotal <- glm(mystops ~ factor(myeth)+factor(myprecinct),family=poisson(link = "log"), data = data)
```

```
AIC(me)
```

```
## [1] 125041.1
```

```
AIC(met)
```

```
## [1] 90483.73
```

```
AIC(mpre)
```

```
## [1] 90905
```

```
AIC(mtotal)
```

```
## [1] 56347.6
```

Based on AIC we can see that by far the model with both ethnicity and precinct (the total model) is the best.

```
BIC(me)
```

```
## [1] 125044.5
```

```
BIC(met)
```

```
## [1] 90493.98
```

```
BIC(mpre)
```

```
## [1] 91161.2
```

```
BIC(mtotal)
```

```
## [1] 56610.64
```

We can also see that based on BIC, once again the model with ethnicity and precinct is the best. It is difficult to write out the model because it has 74 ‘dummy variables’ for precinct and two ‘dummy variables’ for ethnicity. But we can find the expected number of stops in precinct 3 for black people.

```
summary(mtotal)
```

```
##
## Call:
## glm(formula = mystops ~ factor(myeth) + factor(myprecinct), family = poisson(link = "log"),
##      data = data)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)      5.320809   0.051031  104.267 < 2e-16 ***
## factor(myeth)2    -0.447714   0.006061  -73.872 < 2e-16 ***
## factor(myeth)3    -1.414281   0.008558 -165.263 < 2e-16 ***
## factor(myprecinct)2 -0.103919   0.074022  -1.404 0.160352
## factor(myprecinct)3  1.426389   0.056756  25.132 < 2e-16 ***
## factor(myprecinct)4  1.298811   0.057499  22.588 < 2e-16 ***
## factor(myprecinct)5  1.424516   0.056766  25.094 < 2e-16 ***
## factor(myprecinct)6  1.214566   0.058038  20.927 < 2e-16 ***
## factor(myprecinct)7  0.522189   0.064329   8.117 4.76e-16 ***
## factor(myprecinct)8  1.406861   0.056864  24.741 < 2e-16 ***
## factor(myprecinct)9 -0.300754   0.078142  -3.849 0.000119 ***
## factor(myprecinct)10 1.099478   0.058843  18.685 < 2e-16 ***
## factor(myprecinct)11 0.777775   0.061569  12.633 < 2e-16 ***
## factor(myprecinct)12 0.831214   0.061062  13.613 < 2e-16 ***
## factor(myprecinct)13 1.749764   0.055217  31.689 < 2e-16 ***
## factor(myprecinct)14 1.107233   0.058786  18.835 < 2e-16 ***
## factor(myprecinct)15 1.818667   0.054944  33.101 < 2e-16 ***
## factor(myprecinct)16 0.918885   0.060280  15.244 < 2e-16 ***
## factor(myprecinct)17 0.898999   0.060452  14.871 < 2e-16 ***
## factor(myprecinct)18 1.812750   0.054966  32.979 < 2e-16 ***
## factor(myprecinct)19 1.225302   0.057967  21.138 < 2e-16 ***
## factor(myprecinct)20 1.403037   0.056886  24.664 < 2e-16 ***
## factor(myprecinct)21 1.432608   0.056722  25.257 < 2e-16 ***
## factor(myprecinct)22 2.348278   0.053344  44.022 < 2e-16 ***
## factor(myprecinct)23 1.802095   0.055008  32.761 < 2e-16 ***
## factor(myprecinct)24 1.918449   0.054578  35.150 < 2e-16 ***
## factor(myprecinct)25 2.258782   0.053561  42.172 < 2e-16 ***
## factor(myprecinct)26 1.386944   0.056977  24.342 < 2e-16 ***
## factor(myprecinct)27 1.650156   0.055643  29.656 < 2e-16 ***
## factor(myprecinct)28 0.851371   0.060877  13.985 < 2e-16 ***
## factor(myprecinct)29 2.015076   0.054256  37.140 < 2e-16 ***
```

```

## factor(myprecinct)30 1.723230 0.055326 31.147 < 2e-16 ***
## factor(myprecinct)31 1.545073 0.056138 27.523 < 2e-16 ***
## factor(myprecinct)32 0.994694 0.059649 16.676 < 2e-16 ***
## factor(myprecinct)33 2.041726 0.054172 37.690 < 2e-16 ***
## factor(myprecinct)34 1.987696 0.054344 36.576 < 2e-16 ***
## factor(myprecinct)35 0.631548 0.063076 10.012 < 2e-16 ***
## factor(myprecinct)36 1.127625 0.058638 19.230 < 2e-16 ***
## factor(myprecinct)37 0.951507 0.060003 15.858 < 2e-16 ***
## factor(myprecinct)38 1.546733 0.056129 27.557 < 2e-16 ***
## factor(myprecinct)39 1.856298 0.054802 33.873 < 2e-16 ***
## factor(myprecinct)40 0.802526 0.061331 13.085 < 2e-16 ***
## factor(myprecinct)41 1.637104 0.055702 29.390 < 2e-16 ***
## factor(myprecinct)42 1.939956 0.054504 35.593 < 2e-16 ***
## factor(myprecinct)43 1.452252 0.056615 25.651 < 2e-16 ***
## factor(myprecinct)44 1.466737 0.056538 25.942 < 2e-16 ***
## factor(myprecinct)45 2.037334 0.054185 37.599 < 2e-16 ***
## factor(myprecinct)46 2.190855 0.053739 40.769 < 2e-16 ***
## factor(myprecinct)47 0.955511 0.059970 15.933 < 2e-16 ***
## factor(myprecinct)48 1.408132 0.056857 24.766 < 2e-16 ***
## factor(myprecinct)49 0.994694 0.059649 16.676 < 2e-16 ***
## factor(myprecinct)50 2.070964 0.054082 38.293 < 2e-16 ***
## factor(myprecinct)51 1.143478 0.058525 19.538 < 2e-16 ***
## factor(myprecinct)52 1.704039 0.055408 30.755 < 2e-16 ***
## factor(myprecinct)53 1.150079 0.058479 19.667 < 2e-16 ***
## factor(myprecinct)54 0.949499 0.060020 15.820 < 2e-16 ***
## factor(myprecinct)55 1.085540 0.058946 18.416 < 2e-16 ***
## factor(myprecinct)56 0.395896 0.065922 6.006 1.91e-09 ***
## factor(myprecinct)57 0.799027 0.061365 13.021 < 2e-16 ***
## factor(myprecinct)58 1.936965 0.054514 35.531 < 2e-16 ***
## factor(myprecinct)59 1.091662 0.058900 18.534 < 2e-16 ***
## factor(myprecinct)60 2.033262 0.054198 37.515 < 2e-16 ***
## factor(myprecinct)61 1.440020 0.056681 25.405 < 2e-16 ***
## factor(myprecinct)62 1.446155 0.056648 25.529 < 2e-16 ***
## factor(myprecinct)63 1.176054 0.058298 20.173 < 2e-16 ***
## factor(myprecinct)64 1.255357 0.057772 21.730 < 2e-16 ***
## factor(myprecinct)65 1.783064 0.055083 32.371 < 2e-16 ***
## factor(myprecinct)66 2.033262 0.054198 37.515 < 2e-16 ***
## factor(myprecinct)67 1.842403 0.054854 33.588 < 2e-16 ***
## factor(myprecinct)68 1.057971 0.059153 17.885 < 2e-16 ***
## factor(myprecinct)69 1.191164 0.058195 20.468 < 2e-16 ***
## factor(myprecinct)70 1.861156 0.054784 33.973 < 2e-16 ***
## factor(myprecinct)71 2.026780 0.054218 37.382 < 2e-16 ***
## factor(myprecinct)72 2.224272 0.053650 41.459 < 2e-16 ***
## factor(myprecinct)73 2.249239 0.053585 41.975 < 2e-16 ***
## factor(myprecinct)74 1.237433 0.057888 21.376 < 2e-16 ***
## factor(myprecinct)75 -0.178692 0.075517 -2.366 0.017969 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
## Null deviance: 123333 on 224 degrees of freedom
## Residual deviance: 54487 on 148 degrees of freedom
## AIC: 56348

```

```
##  
## Number of Fisher Scoring iterations: 6
```

Given that the base formula without the two dummy variables is for black people, we do not need to use any of the ethnicity variables. But we will need to look at unit 3. This gives us: $\log E(stops) = 5.320809 + 1.426389(1)$. So we expect there to be the log expectation of stops to be 6.747198, or 7 stops.

We can do the same thing for expected number of stops in precinct 3 for white people: $\log E(stops) = 5.320809 + 1.426389(1) - 1.414281$ which tells us the log odds expectation to be 5.332917.

```
5.320809 - 1.414281
```

```
## [1] 3.906528
```

We can answer the same questions for precinct 1. And get that for black people: $\log E(stops) = 5.320809$ giving us (5.320809) and for white people $\log E(stops) = 5.320809 - 1.414281$ we get 3.906528. This is interesting because we can see that the log odds expectation for black people is higher in both precincts than white people.

Part B.

Which model would you prefer based on AIC? Which model would you prefer based on BIC? Would you prefer a model with a baseline or a model without a baseline? Justify your answers. Find out the expected number of stops in precincts 1 and 3 for blacks and whites based on your final choice of model

```
#create the models  
mep <- glm(mystops ~ 1, family = poisson(link = "log"), data = data, offset = log(myarrests))  
metp <- glm(mystops ~ factor(myeth), family = poisson(link = "log"), data = data, offset = log(myarrests))  
mprep <- glm(mystops ~ factor(myprecinct), family = poisson(link = "log"), data = data, offset = log(myarrests))  
mtotalp <- glm(mystops ~ factor(myeth)+factor(myprecinct), family=poisson(link = "log"), data = data, offset = log(myarrests))
```

```
AIC(mep)
```

```
## [1] 47828.87
```

```
AIC(metp)
```

```
## [1] 47149.97
```

```
AIC(mprep)
```

```
## [1] 7752.906
```

```
AIC(mtotalp)
```

```
## [1] 5287.752
```

What we see is that the best model, now that we have offset it by the number of arrests is the full model with both ethnicity and precinct.

```
BIC(mep)
```

```
## [1] 47832.29
```

```
BIC(metp)
```

```
## [1] 47160.21
```

```
BIC(mprep)
```

```
## [1] 8009.113
```

```
BIC(mtotalp)
```

```
## [1] 5550.792
```

What we can see is that again our best model is the full model. We would of course prefer the model with the baseline. This is because our first model assumes that the different precincts are comparable. This is not true. The precincts may have different factors (ethnic breakdown, number of cops, patrolling, etc.) that make them non-comparable! Thus we need to have this baseline that uses the number of arrests so that we can better scale the model to compare the different precincts to each other.

```
summary(mtotalp)
```

```
##
## Call:
## glm(formula = mystops ~ factor(myeth) + factor(myprecinct), family = poisson(link = "log"),
##      data = data, offset = log(myarrests))
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -1.378868   0.051019 -27.027 < 2e-16 ***
## factor(myeth)2     0.010188   0.006802   1.498 0.134190
## factor(myeth)3    -0.419001   0.009435 -44.409 < 2e-16 ***
## factor(myprecinct)2 -0.149050   0.074030  -2.013 0.044077 *
## factor(myprecinct)3  0.559955   0.056758   9.866 < 2e-16 ***
## factor(myprecinct)4  1.210636   0.057549  21.037 < 2e-16 ***
## factor(myprecinct)5  0.282865   0.056794   4.981 6.34e-07 ***
## factor(myprecinct)6  1.144204   0.058047  19.712 < 2e-16 ***
## factor(myprecinct)7  0.218173   0.064335   3.391 0.000696 ***
## factor(myprecinct)8 -0.390565   0.056868  -6.868 6.51e-12 ***
## factor(myprecinct)9  0.485112   0.078151   6.207 5.39e-10 ***
## factor(myprecinct)10 0.417068   0.058851   7.087 1.37e-12 ***
## factor(myprecinct)11 0.654417   0.061586  10.626 < 2e-16 ***
## factor(myprecinct)12 1.148698   0.061067  18.811 < 2e-16 ***
## factor(myprecinct)13 1.053509   0.055250  19.068 < 2e-16 ***
## factor(myprecinct)14 0.614115   0.058803  10.444 < 2e-16 ***
## factor(myprecinct)15 1.054904   0.054956  19.195 < 2e-16 ***
## factor(myprecinct)16 0.797715   0.060290  13.231 < 2e-16 ***
## factor(myprecinct)17 -0.049312   0.060478  -0.815 0.414864
```

```

## factor(myprecinct)18 0.131645 0.054981 2.394 0.016649 *
## factor(myprecinct)19 0.319164 0.057999 5.503 3.74e-08 ***
## factor(myprecinct)20 -0.040574 0.056928 -0.713 0.476016
## factor(myprecinct)21 0.410988 0.056805 7.235 4.65e-13 ***
## factor(myprecinct)22 1.211227 0.053399 22.683 < 2e-16 ***
## factor(myprecinct)23 0.599919 0.055064 10.895 < 2e-16 ***
## factor(myprecinct)24 1.319454 0.054600 24.166 < 2e-16 ***
## factor(myprecinct)25 0.913359 0.053596 17.041 < 2e-16 ***
## factor(myprecinct)26 -0.147614 0.057006 -2.589 0.009612 **
## factor(myprecinct)27 1.895485 0.055664 34.052 < 2e-16 ***
## factor(myprecinct)28 -0.764964 0.060915 -12.558 < 2e-16 ***
## factor(myprecinct)29 1.124710 0.054267 20.726 < 2e-16 ***
## factor(myprecinct)30 0.524652 0.055374 9.475 < 2e-16 ***
## factor(myprecinct)31 1.649498 0.056167 29.368 < 2e-16 ***
## factor(myprecinct)32 1.386276 0.059679 23.229 < 2e-16 ***
## factor(myprecinct)33 1.083192 0.054214 19.980 < 2e-16 ***
## factor(myprecinct)34 1.518972 0.054355 27.945 < 2e-16 ***
## factor(myprecinct)35 0.879407 0.063190 13.917 < 2e-16 ***
## factor(myprecinct)36 1.616306 0.058796 27.490 < 2e-16 ***
## factor(myprecinct)37 1.409430 0.060008 23.487 < 2e-16 ***
## factor(myprecinct)38 1.763813 0.056195 31.388 < 2e-16 ***
## factor(myprecinct)39 0.501081 0.054841 9.137 < 2e-16 ***
## factor(myprecinct)40 1.531984 0.061467 24.924 < 2e-16 ***
## factor(myprecinct)41 1.913898 0.055716 34.351 < 2e-16 ***
## factor(myprecinct)42 1.084217 0.054511 19.890 < 2e-16 ***
## factor(myprecinct)43 0.541787 0.056645 9.565 < 2e-16 ***
## factor(myprecinct)44 0.883038 0.056671 15.582 < 2e-16 ***
## factor(myprecinct)45 0.838439 0.054215 15.465 < 2e-16 ***
## factor(myprecinct)46 0.655855 0.053750 12.202 < 2e-16 ***
## factor(myprecinct)47 1.128144 0.059981 18.808 < 2e-16 ***
## factor(myprecinct)48 0.411880 0.056895 7.239 4.51e-13 ***
## factor(myprecinct)49 1.101219 0.059681 18.452 < 2e-16 ***
## factor(myprecinct)50 0.878797 0.054109 16.241 < 2e-16 ***
## factor(myprecinct)51 0.348980 0.058550 5.960 2.52e-09 ***
## factor(myprecinct)52 0.491898 0.055458 8.870 < 2e-16 ***
## factor(myprecinct)53 0.599687 0.058486 10.254 < 2e-16 ***
## factor(myprecinct)54 0.491840 0.060042 8.192 2.58e-16 ***
## factor(myprecinct)55 0.497511 0.059023 8.429 < 2e-16 ***
## factor(myprecinct)56 0.890877 0.065982 13.502 < 2e-16 ***
## factor(myprecinct)57 1.532782 0.061366 24.978 < 2e-16 ***
## factor(myprecinct)58 1.578846 0.054528 28.955 < 2e-16 ***
## factor(myprecinct)59 1.083726 0.058920 18.393 < 2e-16 ***
## factor(myprecinct)60 0.833828 0.054216 15.380 < 2e-16 ***
## factor(myprecinct)61 1.277336 0.056757 22.506 < 2e-16 ***
## factor(myprecinct)62 1.206378 0.056666 21.289 < 2e-16 ***
## factor(myprecinct)63 1.308782 0.058301 22.449 < 2e-16 ***
## factor(myprecinct)64 1.712497 0.057772 29.642 < 2e-16 ***
## factor(myprecinct)65 1.899204 0.055154 34.434 < 2e-16 ***
## factor(myprecinct)66 2.102128 0.054231 38.763 < 2e-16 ***
## factor(myprecinct)67 1.202841 0.054957 21.887 < 2e-16 ***
## factor(myprecinct)68 2.201257 0.059245 37.155 < 2e-16 ***
## factor(myprecinct)69 1.720700 0.058227 29.551 < 2e-16 ***
## factor(myprecinct)70 1.032004 0.054813 18.828 < 2e-16 ***
## factor(myprecinct)71 1.479415 0.054239 27.276 < 2e-16 ***

```

```
## factor(myprecinct)72 1.464985 0.053737 27.262 < 2e-16 ***
## factor(myprecinct)73 0.991018 0.053585 18.494 < 2e-16 ***
## factor(myprecinct)74 1.151460 0.058023 19.845 < 2e-16 ***
## factor(myprecinct)75 1.571225 0.075731 20.747 < 2e-16 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
## Null deviance: 46120.3 on 224 degrees of freedom
## Residual deviance: 3427.1 on 148 degrees of freedom
## AIC: 5287.8
##
## Number of Fisher Scoring iterations: 4
```

For the expected number of stops in precinct 1 for black people: $\log E(stops) = -1.378868$ Expected number of stops in precinct 1 for white people: $\log E(stops) = -1.378868 - 0.419001 = -1.797869$ Black people in precinct 1: $\log E(stops) = -1.378868 - 0.419001 = -1.797869$ White people in precinct 3: $\log E(stops) = -1.378868 - 0.419001 + 0.559955 = -1.237914$

Part C.

Looking at our final model we can see that ethnicity is indeed statistically significant, which tells us that both of our dummy variables do effect the model. What we can say is that we do have statistically significant evidence that for every single increase in arrest in any given precinct, we expect white people to be pulled over $\log E(-0.419001)$ times less than black people. We also can say that we have statistically significant evidence that for every single increase in arrest in any given precinct (holding precinct constant), based on our model we expect hispanic people to be pulled over $\log E(0.010188)$ times more than black people. This highlights that black and hispanic people do get stopped more times than white people, showing racial bias.

Problem 2

1.

We get the column totals of 46 (Yes) and 1314 (No) and row totals 684 (Placebo) and 658 (Asprin). Our grand total is 1,360.

2.

We will use the subscripts of Y,N for the yes and now columns and P,A for placebo and asprin rows. We get the following upper and lower bounds:

$$\max\{0\} \leq n_{yp} \leq \min\{46\} \quad \max\{0\} \leq n_{ya} \leq \min\{46\} \quad \max\{638\} \leq n_{np} \leq \min\{684\} \quad \max\{630\} \leq n_{na} \leq \min\{676\}$$

3.

We know that the number of tables is the upper bound minus the lower bound. This is the same across the four Facht bounds and is 46 tables. We can say the count in the (1,1) tables is x and thus we can set $T = \{(x, 46 - x, 684 - x, 676 - x) : x \in \{0, 1, \dots, 46\}\}$.

4.

```
#create the table
myocardial = c(28, 18, 656, 658)
myocardial.array = array(myocardial, c(2,2))
#create saturated model
sat_model = loglin(myocardial.array, margin=list(c(1,2)),fit=TRUE,param=TRUE)
```

```
## 2 iterations: deviation 0
```

```
sat_model
```

```
## $lrt
## [1] 0
##
## $pearson
## [1] 0
##
## $df
## [1] 0
##
## $margin
## $margin[[1]]
## [1] 1 2
##
##
## $fit
##      [,1] [,2]
## [1,]   28 656
## [2,]   18 658
##
## $param
## $param$'(Intercept) '
## [1] 4.799485
##
## $param$'1'
## [1] 0.1096972 -0.1096972
##
## $param$'2'
## [1] -1.688197 1.688197
##
## $param$'1.2'
##      [,1]      [,2]
## [1,] 0.1112192 -0.1112192
## [2,] -0.1112192 0.1112192
```

This gives us $u = 4.799485, u_{1(1)} = 0.1096972, u_{1(2)} = -0.1096972, u_{2(1)} = -1.688197, u_{2(2)} = 1.688197, u_{12(12)} = 0.1112192$.

5.

To find the expected cell values under the log-linear model of independence of Aspirin Use and Myocardial Infarction we can use the below code:

```
indepmodel = loglin(myocardial.array, margin=list(1,2),fit=TRUE,param=TRUE)
```

```
## 2 iterations: deviation 0
```

```
indepmodel
```

```
## $lrt
## [1] 2.147353
##
## $pearson
## [1] 2.129972
##
## $df
## [1] 1
##
## $margin
## $margin[[1]]
## [1] 1
##
## $margin[[2]]
## [1] 2
##
##
## $fit
##           [,1]      [,2]
## [1,] 23.13529 660.8647
## [2,] 22.86471 653.1353
##
## $param
## $param$'(Intercept)'
## [1] 4.811572
##
## $param$'1'
## [1] 0.005882421 -0.005882421
##
## $param$'2'
## [1] -1.676095 1.676095
```

This gives us $u = 4.811572$, $u_{1(1)} = 0.005882421$, $u_{1(2)} = -0.005882421$, $u_{2(1)} = -1.676095$, $u_{2(2)} = 1.676095$, and we do not have a $u_{12(12)}$ because we are assuming independence. We can also see the expected m_{ij} are 23.13529, 660.8647, 22.86471, and 653.1353 (reading left to right).

6.

Using the code below we can perform an asymptotic test of independence of Aspirin Use and Myocardial Infarction based on Pearson's chi-square statistic:

```
1-pchisq(indepmodel$pearson-sat_model$pearson, indepmodel$df-sat_model$df)
```

```
## [1] 0.1444434
```

We can see that the chi-square statistic is far above the alpha of .05.

7.

Using the following code we can perform an asymptotic test of independence of Aspirin Use and Myocardial Infarction based on the likelihood ratio statistic:

```
1-pchisq(indepmodel$lrt-sat_model$lrt, indepmodel$df-sat_model$df)
```

```
## [1] 0.1428159
```

We can see that once again, the chi-squared statistic is far above the alpha level of .05.

8.

We are testing $H_0 : \alpha = 1$ versus $H_A : \alpha \neq 1$. To test this we can perform Fisher's exact test of independence:

```
fisher.test(myocardial.array, simulate.p.value=TRUE, B=1e5)
```

```
##
## Fisher's Exact Test for Count Data
##
## data: myocardial.array
## p-value = 0.1768
## alternative hypothesis: true odds ratio is not equal to 1
## 95 percent confidence interval:
## 0.8236804 3.0256940
## sample estimates:
## odds ratio
## 1.559785
```

We can see that we fail to reject the null hypothesis, thus we do not have statistically significant evidence to suggest that there is interaction between Aspirin and Myocardial Infarction.

9.

I do not know how to do this one but I appreciate you grader! I tried my best on the rest of the hw!

10.

Due to p-values that are above our alpha level of .05 for both chi-squared tests, we do not see any statistically significant effect of Aspirin on occurrence of Myocardial Infarction.

Problem #3a.

Prove that IPF converges after one iteration to the expected cell counts.

$$m_{ij} = \frac{n_{i+} \cdot n_{+j}}{n_{++}} \quad \text{for all } 1 \leq i \leq 2, 1 \leq j \leq 2$$

The purpose of IPF is to turn separate one-dimensional data into a single two-dimensional table. What it does is

Continually updating an initial seed with respect to the marginal n_{i+} and then n_{+j} . So what we are

doing is providing one row and one column adjustment.

We can start with our 2×2 table as initialized $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ then we adjust by the row, giving us

$$\begin{pmatrix} \frac{n_{1+}}{2} & \frac{n_{1+}}{2} \\ \frac{n_{2+}}{2} & \frac{n_{2+}}{2} \end{pmatrix} \quad \text{but now we need to do the same thing to adjust by columns. We end up with our first iterations of } m_{ij} = \frac{n_{i+}}{2} \quad \text{and} \quad n_{+j} = \frac{n_{++}}{2}$$

$$\text{This gives us the first iteration of } m_{ij} = \frac{n_{ij}}{\left(\frac{n_{++}}{2}\right)} \cdot \frac{n_{i+}}{2} = n_{+j} \times \frac{2}{n_{++}} \cdot \frac{n_{i+}}{2} = \frac{n_{+j} \cdot n_{i+}}{n_{++}}$$

#3b. Prove that the expected cell counts in equation $m_{ij} = \frac{n_{i+} \cdot n_{+j}}{n_{++}}$ satisfy:

$$m_{i+} = n_{i+} \quad \text{for all } 1 \leq i \leq 2$$

$$m_{+j} = n_{+j} \quad \text{for all } 1 \leq j \leq 2$$

We want to check $m_{i+} = n_{i+}$ we know i takes on 1, 2 so we want $m_{1+} = n_{1+}$ and $m_{2+} = n_{2+}$, along with $m_{+1} = n_{+1}$ and $m_{+2} = n_{+2}$. Plugging this into our equation we get:

$$m_{11} = \frac{n_{1+} \cdot n_{+1}}{n_{++}}, \quad \text{which makes sense! we can do the same thing iterating through}$$

every row and column and will get the same results!